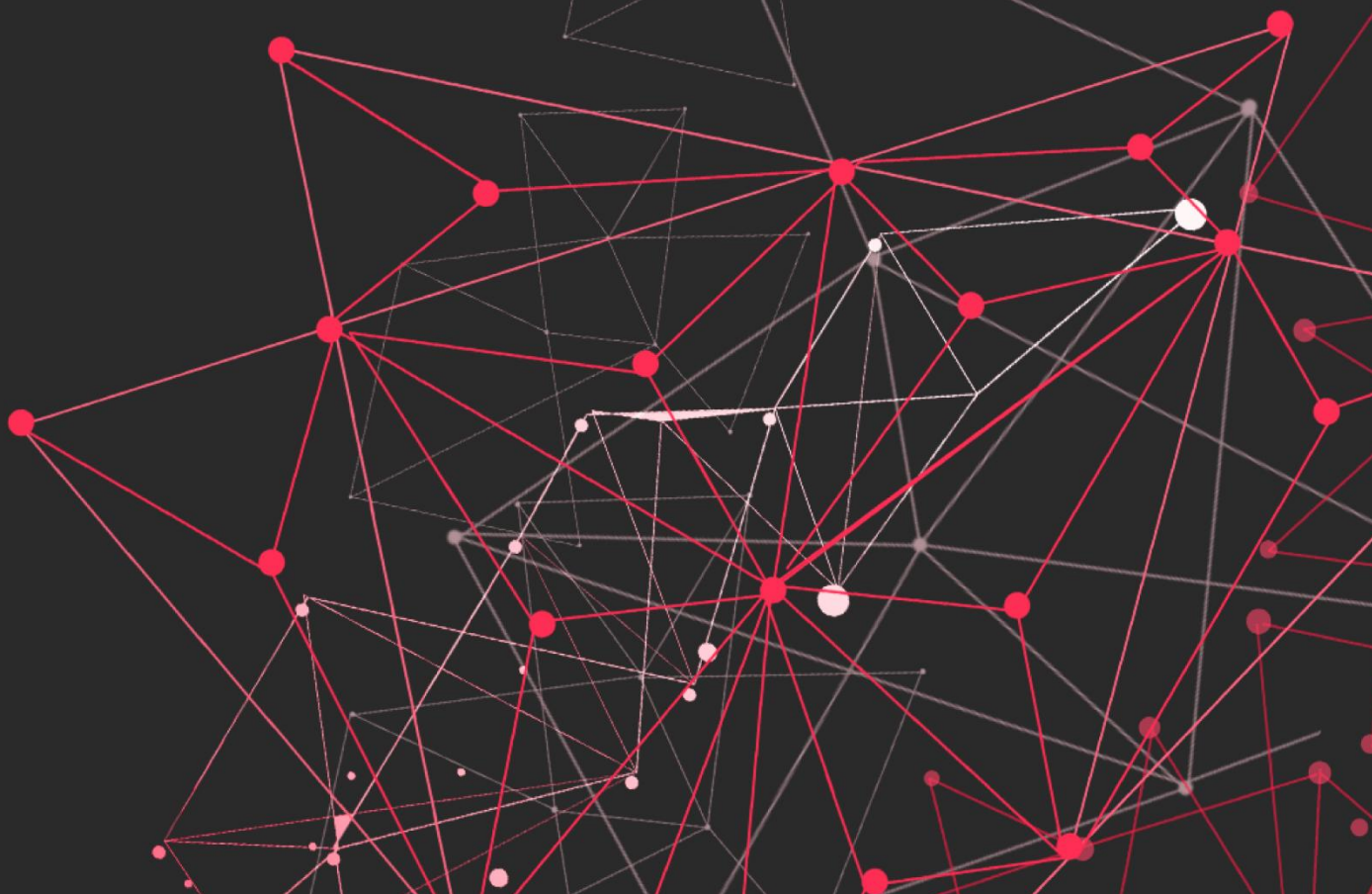




# ИНСТРУКЦИЯ ПО GIT GITHUB В MARKDOWN

Практическое руководство по  
системе контроля версий Git,  
платформе GitHub и языку  
разметки Markdown





## Оглавление

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. Установка и первичная настройка Git.....</b>       | <b>3</b>  |
| 1.1. Проверка наличия Git .....                          | 3         |
| 1.2. Установка Git .....                                 | 3         |
| 1.3. Настройка пользователя .....                        | 3         |
| <b>2. Основы работы с локальным репозиторием .....</b>   | <b>4</b>  |
| 2.1. Инициализация репозитория .....                     | 4         |
| 2.2. Добавление файлов и создание коммитов .....         | 4         |
| 2.3. Просмотр состояния и истории .....                  | 6         |
| 2.4. Перемещение между коммитами.....                    | 7         |
| 2.5. Сравнение изменений.....                            | 8         |
| 2.6. Игнорирование файлов .....                          | 8         |
| <b>3. Работа с ветками.....</b>                          | <b>9</b>  |
| 3.1. Создание и переключение веток .....                 | 9         |
| 3.2. Просмотр веток .....                                | 10        |
| 3.3. Слияние веток .....                                 | 10        |
| 3.4. Конфликты при слиянии.....                          | 11        |
| 3.5. Удаление веток .....                                | 11        |
| 3.6. Восстановление потерянных коммитов (reflog) .....   | 12        |
| <b>4. Работа с удалённым репозиторием (GitHub) .....</b> | <b>12</b> |
| 4.1. Клонирование существующего репозитория.....         | 12        |
| 4.2. Получение изменений с сервера.....                  | 13        |
| 4.3. Отправка изменений на сервер .....                  | 13        |
| 4.4. Полный цикл совместной работы (GitHub) .....        | 14        |
| 4.5. Pull Request.....                                   | 14        |
| <b>5. Краткий справочник по Markdown .....</b>           | <b>16</b> |



## 1. Установка и первичная настройка Git

### 1.1. Проверка наличия Git

Откройте терминал и выполните:

```
Bash  
1  git --version
```

Если Git установлен, отобразится версия (например, git version 2.45.0). В противном случае появится сообщение об ошибке.

### 1.2. Установка Git

Скачайте последнюю версию с официального сайта: <https://git-scm.com/downloads>  
Устанавливайте с настройками по умолчанию.

### 1.3. Настройка пользователя

При первом использовании Git нужно представиться. Выполните в терминале:

```
Bash  
1  git config --global user.name "Ваше Имя"  
2  git config --global user.email "ваша_почта@example.com"
```

**Совет:** начиная с Git 2.28 основная ветка по умолчанию называется main, а не master. Чтобы сразу задать это имя для всех новых репозиториях, выполните:

```
Bash  
1  git config --global init.defaultBranch main
```



## 2. Основы работы с локальным репозиторием

### 2.1. Инициализация репозитория

Перейдите в папку проекта и выполните:

```
● ● ● Bash  
1 git init
```

В папке появится скрытый каталог `.git` — репозиторий создан.

### 2.2. Добавление файлов и создание коммитов

- **Добавить конкретный файл** в индекс (подготовить к коммиту):

```
● ● ● Bash  
1 git add <имя_файла>
```

- **Добавить все изменения** в текущей папке и подпапках (новые и изменённые файлы, но не удалённые):

```
● ● ● Bash  
1 git add .
```



- **Добавить вообще всё** (включая удалённые файлы):



Bash

```
1 git add -A
```

- **Создать коммит** с сообщением:



Bash

```
1 git commit -m "Осмысленный комментарий к изменениям"
```

- **Объединённая команда** (добавляет в коммит все изменения отслеживаемых файлов и создаёт коммит):



Bash

```
1 git commit -a -m "Комментарий"
```

Важно: -a не добавляет новые, ещё не отслеживаемые файлы. Для новых файлов предварительно нужен git add.



## 2.3. Просмотр состояния и истории

- **Текущее состояние репозитория** (какие файлы изменены, добавлены, не отслеживаются):



Bash

```
1 git status
```

- **История коммитов** с хешами, авторами и датами:



Bash

```
1 git log
```

Для компактного вывода:



markdown

```
1 git log --online
```



# git

## 2.4. Перемещение между коммитами

- Перейти к состоянию определённого коммита (режим просмотра, «detached HEAD»):

```
● ● ● Bash
1 git checkout <хеш_коммита>
```

- Вернуться к актуальной ветке (раньше master, сейчас чаще main):

```
● ● ● Bash
1 git checkout main
```

**Новый подход:** начиная с Git 2.23 для переключения веток удобнее использовать `git switch`, а для отмены изменений в файлах - `git restore`. Однако `checkout` полностью рабочий и остаётся в обиходе.

```
● ● ● Bash
1 git switch main          # переключиться на ветку main
2 git restore <файл>      # отменить изменения в файле
```





## 2.5. Сравнение изменений

Просмотр различий между текущим состоянием файлов и последним коммитом:

```
● ● ● Bash
1  git diff
```

Можно сравнивать конкретные коммиты: `git diff <хеш1> <хеш2>`.

## 2.6. Игнорирование файлов

Создайте в корне репозитория файл `.gitignore` и пропишите в нём имена или шаблоны файлов/папок, которые Git не должен отслеживать. Пример содержимого:

```
● ● ● Bash
1  *.log
2  node_modules/
3  secret.txt
```





# git

## 3. Работа с ветками

### 3.1. Создание и переключение веток

- Создать ветку:

```
● ● ● Bash  
1 git branch <имя_новой_ветки>
```

- Переключиться на ветку:

```
● ● ● Bash  
1 git checkout <имя_ветки>
```

- Создать и сразу переключиться (одна команда):

```
● ● ● Bash  
1 git checkout -b <имя_новой_ветки>
```

Современный аналог: `git switch -c <имя_новой_ветки>`.



### 3.2. Просмотр веток

Список локальных веток (текущая отмечена \*):

```
● ● ● Bash
1  git branch
```

Графическая история веток:

```
● ● ● Bash
1  git log --graph --oneline
```

### 3.3. Слияние веток

Находясь в целевой ветке (например, main), выполните:

```
● ● ● Bash
1  git merge <имя_вливаемой_ветки>
```

Изменения из указанной ветки будут добавлены в текущую.



### 3.4. Конфликты при слиянии

Если один и тот же фрагмент текста/кода изменён в обеих сливаемых ветках, Git сообщит о конфликте.

В конфликтном файле появятся маркеры <<<<<<, =====, >>>>>>.

Вручную выберите нужный вариант и удалите маркеры. Возможные стратегии:

- Оставить текущее изменение (то, что было в активной ветке до слияния).
- Принять входящее изменение (из сливаемой ветки).
- Оставить оба варианта.
- Сравнить и отредактировать вручную.

После разрешения конфликта обязательно выполните:

```
Bash

1  git add <разрешённый_файл>
2  git commit -m "Разрешён конфликт"
```

### 3.5. Удаление веток

- **Безопасное удаление** (если ветка полностью слита):

```
Bash

1  git branch -d <имя_ветки>
```



- **Принудительное удаление** (даже если есть неслитые изменения):

```
Bash
1  git branch -D <имя_ветки>
```

Примечание: нельзя удалить ветку, в которой вы находитесь. Сначала переключитесь на другую (`git switch main`).

### 3.6. Восстановление потерянных коммитов (reflog)

Журнал всех перемещений HEAD (даже удалённые ветки и откаты):

```
Bash
1  git reflog
```

Позволяет найти хеш коммита, на который можно затем переключиться или восстановить ветку. История хранится локально и не зависит от коммитов в `git log`.

## 4. Работа с удалённым репозиторием (GitHub)

### 4.1. Клонирование существующего репозитория

Скопировать удалённый репозиторий на свой компьютер:

```
Bash
1  git clone <URL_репозитория>
```



Перейти в папку клонированного проекта:

```
● ● ● Bash
1 cd <имя_папки_репозитория>
```

## 4.2. Получение изменений с сервера

- Скачать и автоматически слить с текущей локальной веткой:

```
● ● ● Bash
1 git pull
```

Для основной ветки обычно: `git pull origin main`.

## 4.3. Отправка изменений на сервер

- Первый пуш (связывает локальную ветку с удалённой):

```
● ● ● Bash
1 git push -u origin main
```

Флаг `-u` запоминает связь, и в дальнейшем достаточно просто `git push`.



- Последующие отправки:



Bash

```
1 git push
```

При первом использовании потребуется авторизация (токен или логин/пароль).

#### 4.4. Полный цикл совместной работы (GitHub)

1. Создайте аккаунт на **GitHub.com**.
2. Создайте локальный репозиторий или склонируйте существующий.
3. «Подружите» локальный и удалённый репозиторий (GitHub при создании нового репозитория подскажет команды).
4. Отправьте изменения на GitHub командой `git push -u origin main`.
5. Другие участники или вы с другого компьютера клонируют репозиторий и вносят изменения.
6. Чтобы получить актуальное состояние, используйте `git pull`.

#### 4.5. Pull Request

**Pull Request** — это запрос на вливание ваших изменений в основной репозиторий.

**Алгоритм создания Pull Request на GitHub:**

1. Сделайте **fork** (ответвление) целевого репозитория — появится его копия в вашем аккаунте.
2. Склонируйте **свою** версию репозитория на компьютер:



Bash

```
1 git clone <URL_вашего_форка>
```



3. Создайте новую ветку для своих изменений:



Bash

```
1 git checkout -b feature/моё-улучшение
```

4. Внесите изменения, зафиксируйте их (коммиты).
5. Отправьте ветку в свой удалённый репозиторий:



Bash

```
1 git push -u origin feature/моё-улучшение
```

6. Зайдите на GitHub, перейдите в свой форк и нажмите кнопку **Pull Request**. Опишите, что и зачем вы изменили.





## 5. Краткий справочник по Markdown

Markdown - облегчённый язык разметки. Ссылка на подробное руководство: [Microsoft](#)

Базовый синтаксис:

- **Заголовки** — символ #. Количество # задаёт уровень (1–6):

```
markdown

1  # Заголовок 1
2  ## Заголовок 2
3  ##### Заголовок 6
```

Альтернативное выделение заголовков 1 и 2 уровней (не менее трёх символов подчёркивания):

```
markdown

1  Заголовок 1
2  =====
3  Заголовок 2
4  -----
```

- **Полужирный текст** — две звёздочки без пробелов: **\*\*текст\*\***
- *Курсив* — одна звёздочка: *\*текст\**
- **Полужирный курсив** — три звёздочки: **\*\*\*текст\*\*\***
- *Зачёркнутый текст* — две тильды: *~~текст~~*



- **Маркированный список** — \*, - или + в начале строки:



markdown

- 1 \* Элемент
- 2 \* Элемент

- **Нумерованный список** — цифра с точкой:



markdown

- 1 1. Первый
- 2 2. Второй